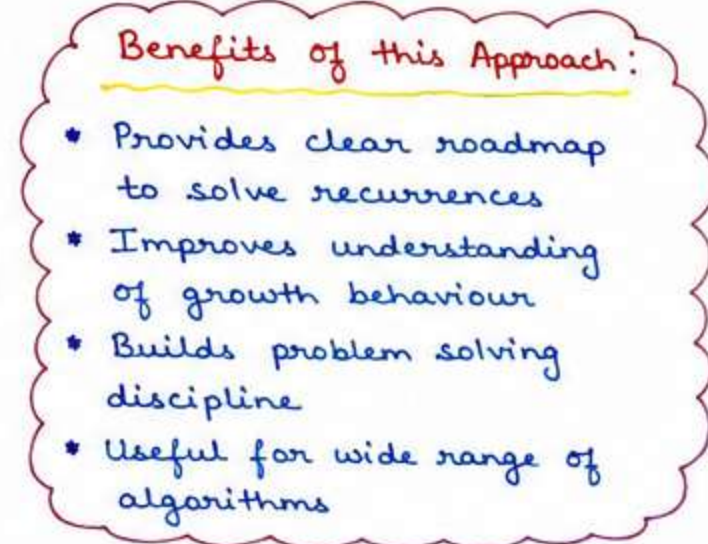
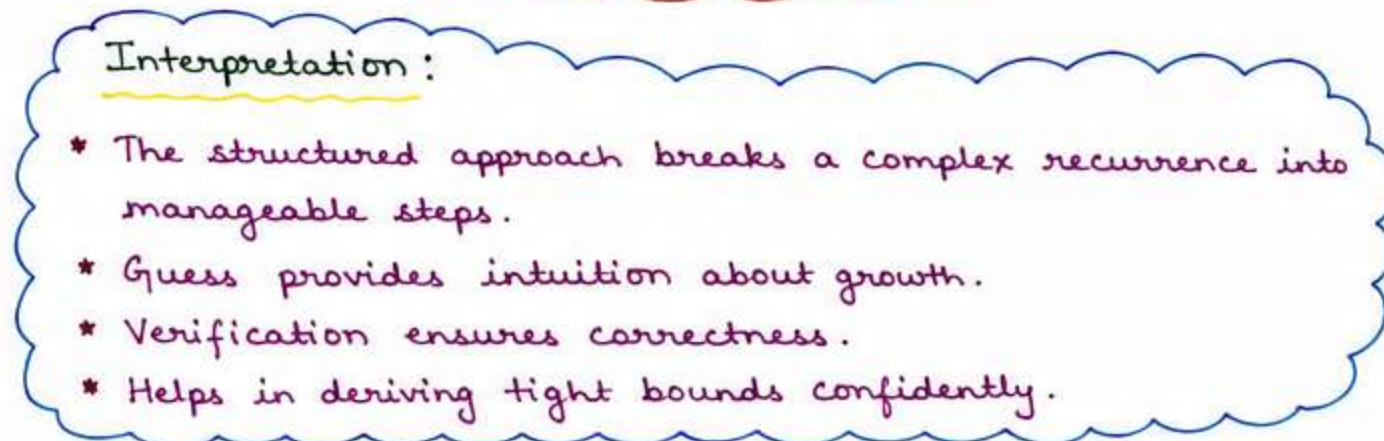
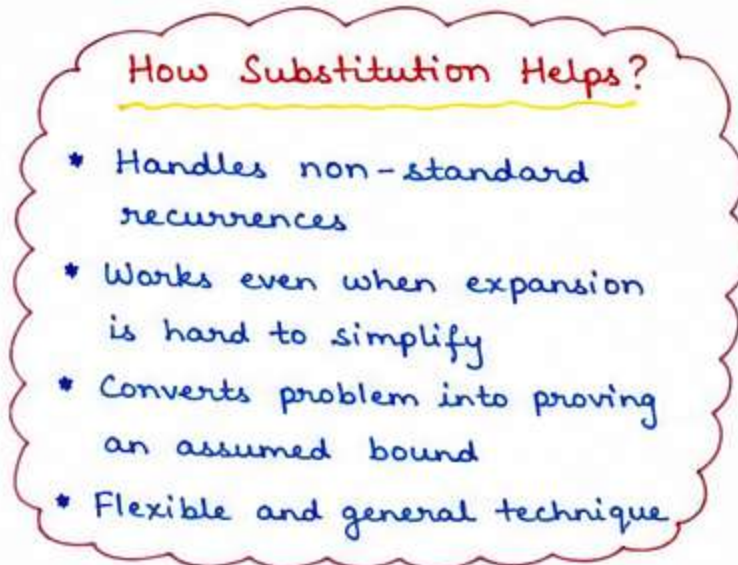
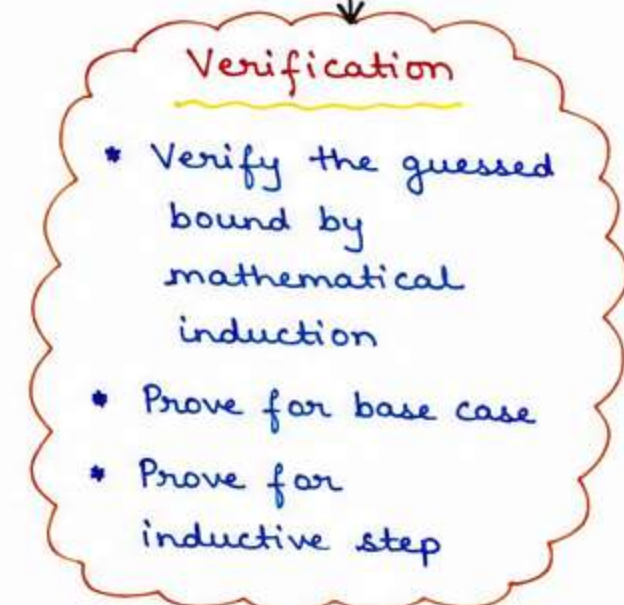
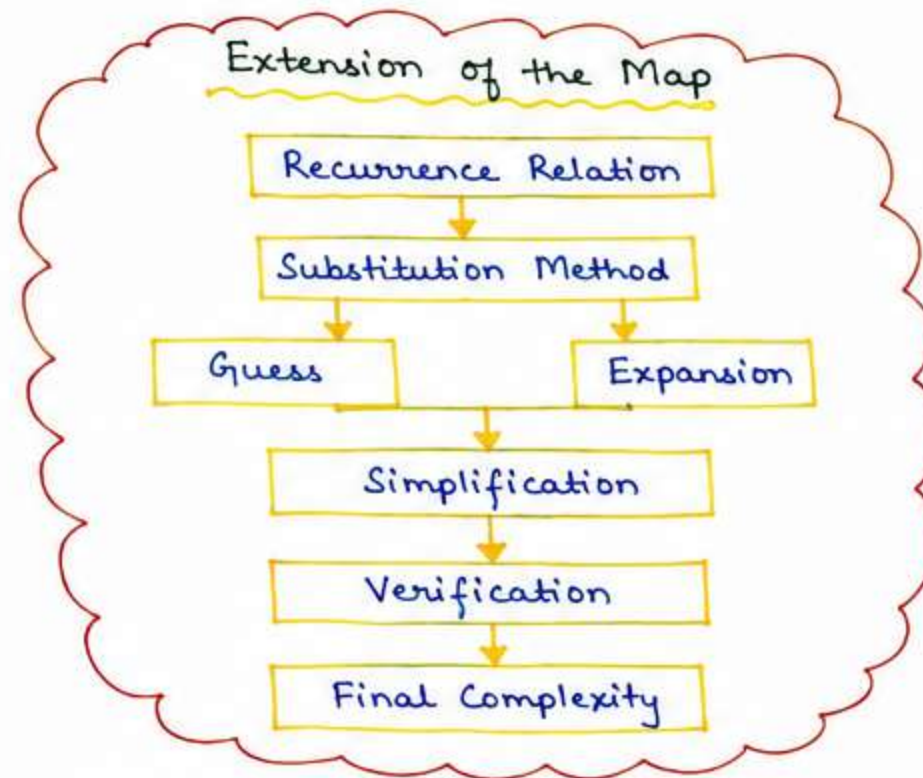
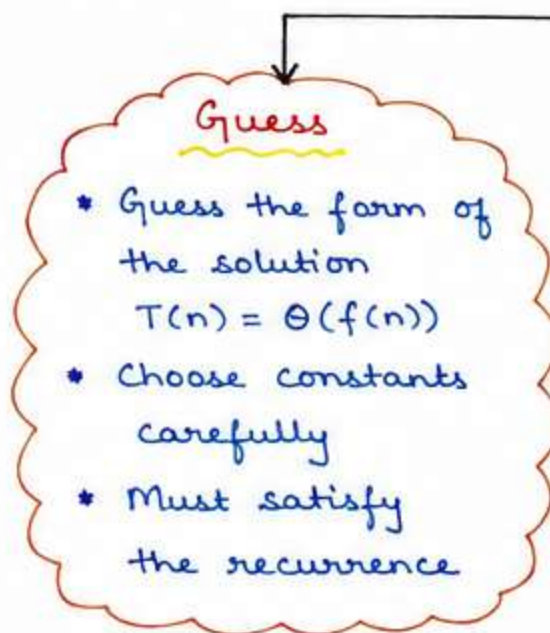
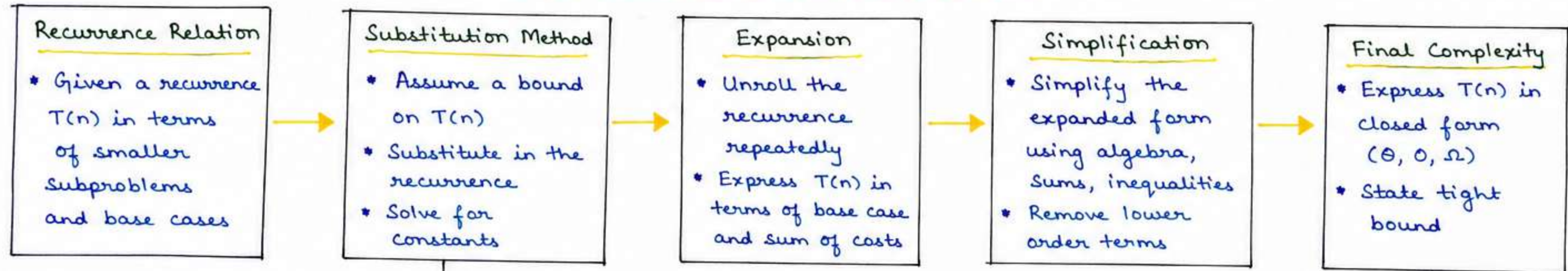


Q) 56

Recurrence Relation $\rightarrow$ Substitution Method $\rightarrow$ Expansion $\rightarrow$ Simplification $\rightarrow$ Final Complexity
(Extend with Guess and Verification)



Q16.

Algorithm $\rightarrow$ Operation Count $\rightarrow$ Growth Function $\rightarrow$ Big-O Notation $\rightarrow$ Upper Bound
(Extend with Worst-case Analysis)

Algorithm

- A step-by-step procedure to solve a problem
- Input size = n

Operation Count

- Count the number of basic operations executed
- Depends on input size (n)

Growth Function

- Express operation count as a function of n
- $f(n)$ represents the running time

Big-O Notation

- Gives an asymptotic upper bound
- $f(n) = O(g(n))$
- Ignores constants and lower order terms

Upper Bound

- Represents the maximum growth rate
- $T(n) \leq c \cdot g(n)$ for $n \geq n_0$

Why Big-O Represents an Upper Limit?

- It bounds the running time from above
- Exact count may vary with different inputs
- Big-O gives a safe guarantee
- Independent of machine and implementation details

Worst-case Analysis

- Consider the input that causes maximum operations
- Provides the largest running time
- Basis for Big-O

How the Structure Helps?

- Clear path from algorithm to performance bound
- Worst-case ensures reliability
- Big-O allows easy comparison between algorithms
- Useful for analyzing efficiency for large inputs
- Supports decision making in choosing better algorithms

Interpretation :

- * The completed map shows how we move from counting basic operations to a guaranteed upper bound on running time.
- * Worst-case analysis supports the Big-O notation by focusing on the maximum possible time.
- * This structure helps us estimate performance reliably, especially for large input sizes, where exact count is impractical.